International Journal of Information Security (2026) 25:174
https://doi.org/10.1007/s10207-026-01342-x

REGULAR CONTRIBUTION

# WSymBERT: a weighted neuro-symbolic model for interpretable CVSS prediction

**Seyedeh Leili Mirtaheri[1] · Amirhossein Majd[1] · Reza Shahbazian[2] · Georgios Spathoulas[3] · Aida Akbarzadeh[3] · Andrea Pugliese[1]**

Received: 15 May 2026 / Accepted: 26 September 2026
© The Author(s) 2026

**Abstract**

The rapid growth in reported software vulnerabilities, combined with substantial delays in manual Common Vulnerability Scoring System (CVSS) assessment, has created a critical need for automated and interpretable vulnerability scoring tools. When vulnerabilities remain unscored for long periods, organizations face greater difficulty in prioritizing patches and mitigation actions, which can leave exploitable weaknesses insufficiently addressed during a critical exposure window. This challenge is particularly important for the CVSS Base metrics, which form the foundation for severity assessment and are essential for timely risk evaluation. This paper presents WSymBERT, a neuro-symbolic approach for predicting CVSS v3.1 Base metrics by combining contextual language representations with expert-derived symbolic guidance through a symbolic-guided weighted pooling mechanism. We evaluate WSymBERT on two benchmark datasets, achieving mean accuracies of 96.2% on NVD and 94.7% on Combine across the eight CVSS Base metrics. Across the reported evaluations, WSymBERT achieved higher observed performance than the evaluated baselines, while attribution-based analyses indicated greater emphasis on security-relevant evidence under the specified evaluation protocol.

**Keywords** Vulnerability scoring · CVSS prediction · Neuro-symbolic · Natural language processing · Explainable AI · Information security

## 1 Introduction

The number of disclosed software vulnerabilities has grown rapidly in recent years, placing increasing pressure on security teams to assess risk and prioritize remediation in a timely manner. At the same time, vulnerability scoring in practice still depends heavily on manual expert analysis [1]. This creates a serious operational bottleneck: many newly published vulnerabilities remain without complete Common Vulnerability Scoring System (CVSS) assessments for extended periods [2], limiting their usefulness for patch prioritization, vulnerability triage, and downstream risk management. When vulnerabilities remain unscored or are scored late, organizations may struggle to identify which issues require urgent action, leaving exploitable weaknesses exposed during a critical response window. The CVSS is the most widely used standard for communicating vulnerability severity in a structured and comparable form [3]. In particular, the CVSS v3.1 Base metrics capture the core exploitability and impact characteristics of a vulnerability and serve as the foundation for further severity analysis. However,

✉ Seyedeh Leili Mirtaheri
leili.mirtaheri@dimes.unical.it

Amirhossein Majd
amir.majd@dimes.unical.it

Reza Shahbazian
reza.shahbazian@unipa.it

Georgios Spathoulas
georgios.spathoulas@ntnu.no

Aida Akbarzadeh
aida.akbarzadeh@ntnu.no

Andrea Pugliese
andrea.pugliese@unical.it

[1] Department of Informatics, Modeling, Electronics and System Engineering (DIMES), University of Calabria, 87036 Arcavacata di Rende, CS, Italy

[2] Department of Humanities, University of Palermo, 90128 Palermo, PA, Italy

[3] Department of Information Security and Communication Technology, Norwegian University of Science and Technology (NTNU), Gjøvik 2815, Norway

Published online: 11 October 2026

assigning these metrics requires interpreting short and often ambiguous natural-language vulnerability descriptions and mapping them to discrete security-relevant categories. This process is time-consuming, expert-dependent, and difficult to scale as the volume of reported vulnerabilities continues to increase. These challenges have motivated growing interest in automated CVSS prediction. Earlier approaches relied on classical machine learning models with manually engineered textual features, while more recent studies have adopted transformer-based language models to learn contextual representations from vulnerability descriptions. Although these methods have improved predictive performance, important limitations remain. In particular, many existing models operate as black boxes: they can produce accurate predictions, but they do not make their reasoning process sufficiently transparent for operational cybersecurity settings, where analysts often need to justify severity decisions. In addition, post-hoc explainability methods may highlight influential words, but they do not ensure that the model reasons in a way that is consistent with expert knowledge or CVSS semantics. This gap is especially important in vulnerability assessment, where prediction quality alone is not enough. A useful scoring system should not only classify vulnerability characteristics accurately but also provide explanations that are aligned with domain logic and traceable to meaningful evidence in the text. This calls for approaches that go beyond purely data-driven pattern matching and incorporate structured security knowledge directly into the learning process.

### 1.1 Problem definition

To bridge the gap between black-box prediction and domain-aware reasoning, we formulate the task as a neuro-symbolic mapping problem. A software vulnerability is typically reported as an unstructured natural language text detailing its cause, prerequisites, and potential impact. Translating this unstructured description into a standardized severity score requires understanding specific technical characteristics. The CVSS formally captures these characteristics using a structured vector as a standardized, machine-readable text string, representing the values assigned to various metrics. For CVSS version 3.1, this representation depends on eight fundamental Base metrics. Let $d$ denote a textual vulnerability description (e.g., see Example 1). The goal is to predict the corresponding CVSS Base metric vector $\mathbf{M} = \{AV, AC, PR, UI, S, C, I, A\}$, where each component is a categorical variable with discrete values defined in the CVSS specification.

Specifically, these metrics define the *Exploitability* factors, including the Attack Vector (AV), Attack Complexity (AC), Privileges Required (PR), and User Interaction (UI). The Scope (S) metric determines whether a vulnerability in one component impacts resources in another. Finally, the *Impact* factors including Confidentiality (C), Integrity (I), and Availability (A), measure the consequences of a successful exploit on the affected system. A typical CVSS vector format resembles a string such as `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H`, detailing the exact categorical value for each metric.

Once $\mathbf{M}$ is known, the numerical Base Score (ranging from 0.0 to 10.0) can be deterministically computed using established equations (see Table 1) [3]. The learning objective is therefore to approximate a mapping, $f : d \rightarrow \mathbf{M}$ that is both accurate (matching expert-assigned labels) and interpretable (consistent with domain reasoning). Unlike traditional approaches that rely solely on textual features, this formulation explicitly requires capturing the semantic meaning of unstructured text while enforcing the symbolic constraints of the CVSS model. This study addresses these challenges through WSymBERT, a hybrid neuro-symbolic model that predicts all eight CVSS v3.1 Base metrics from text. The model integrates expert- and model-derived symbolic weighting into a learned pooling mechanism, enabling domain knowledge to influence the representation used for metric-specific prediction.

**Example 1** Let $d$ denote a textual vulnerability description. For instance, consider the following description for CVE-2022-33955:

> *"IBM CICS TX 11.1 could allow an attacker with physical access to the system to execute code due using* [sic] *a back and refresh attack."*

The official severity assessment for this vulnerability is as follows:

- CVE ID: CVE-2022-33955
- CVSS v3.1 Base Score: 4.3 (Medium)
- CVSS Vector: `CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L`

### 1.2 Contributions

To address this need, we propose WSymBERT, a neuro-symbolic framework for automated prediction of the eight CVSS v3.1 Base metrics from vulnerability descriptions.

- We formulate CVSS metric prediction using eight independently trained metric-specific WSymBERT models.
- We introduce a symbolic-guided weighted pooling mechanism that combines learned token-level pooling with expert- and KeyBERT-derived symbolic weighting.
- We evaluate the framework on the NVD and Combine datasets and report metric-wise predictive performance under the stated experimental protocols.

**Table 1** CVSS v3.1 base score calculation formulas

| Metric | Equation |
|---|---|
| ISS (impact sub-score) | $1 - [(1 - C) \times (1 - I) \times (1 - A)]$ |
| Impact (scope unchanged) | $6.42 \times \text{ISS}$ |
| Impact (scope changed) | $7.52 \times (\text{ISS} - 0.029) - 3.25 \times (\text{ISS} - 0.02)^{1.5}$ |
| Exploitability | $8.22 \times AV \times AC \times PR \times UI$ |
| Base score (impact $\leq 0$) | 0 |
| Base score (scope unchanged) | Roundup(min[(Impact + Exploitability), 10]) |
| Base score (scope changed) | Roundup(min[$1.08 \times$ (Impact + Exploitability), 10]) |

– We analyze attribution behavior using Integrated Gradients and complementary perturbation-based metrics, without treating pooling weights themselves as causal explanations.

The rest of the paper is organized as follows. Section 2 reviews prior work on automated CVSS prediction, explainable AI, and neuro-symbolic learning. The proposed WSymBERT architecture is presented in Sect. 3. Section 4 describes the datasets, experimental setup, and evaluation results. Finally, Sect. 5 concludes the paper and outlines future directions.

## 2 Related work

Research on automated CVSS prediction has evolved from simple text-based methods to advanced neural and hybrid systems. The main directions include classical machine learning (ML), transformers, symbolic and neuro-symbolic models, and studies that integrate explainability or external data sources.

### 2.1 Classical ML approaches

Early studies used traditional machine learning to predict CVSS scores from text. These models represented vulnerability descriptions as word frequency vectors using *Bag of Words* or *TF–IDF* features and then applied algorithms such as support vector machines, random forests, and XGBoost [4–6].

Wang et al. [5] applied XGBoost to predict severity levels and achieved around 92% accuracy but noted that results were strongly dependent on surface-level word matches. Chen et al. [6] used linear regression models that offered transparency through visible features but lacked the capability to capture semantic meaning or sentence structure.

Because these models ignored context and word order, they often failed on metrics that required deeper understanding, such as *Attack Vector (AV)* or *Privileges Required (PR)*. They were interpretable but limited in accuracy, showing a clear need for models that could capture semantics in vulnerability descriptions.

### 2.2 Transformer-based models

The next generation of research adopted transformer models such as BERT and SBERT to process text at a contextual level [7–9]. Each CVSS metric was treated as a separate classification problem.

The CVSS-BERT framework by Shahid and Debar trained one BERT model per metric and visualized relevant words using gradient-based saliency maps [10]. Later, SecureBERT and CVEDrill extended this approach and achieved higher accuracy across most metrics [8]. DistilBERT was also used to reduce model size and training time while maintaining similar performance [11].

Recent studies have further explored task-specific and generative architectures for automated vulnerability scoring. Mirtaheri and Pugliese [12] used generative AI to enrich vulnerability descriptions before transformer-based CVSS prediction. Mirtaheri et al. [13] investigated a fine-tuned BERT–CNN architecture for automated vulnerability scoring. More recently, Mirtaheri et al. [14] combined LLM-based data augmentation with a lightweight fine-tuned BERT model to target an accuracy–computational-cost trade-off.

Despite these advances, transformer models remain opaque. They often behave as black boxes and may base decisions on statistical shortcuts rather than real security semantics [15, 16]. Even internal attention patterns may not represent true reasoning, as shown by the "Attention is not Explanation" study [17]. As a result, these models are hard for analysts to trust in critical security settings.

### 2.3 Symbolic and neuro-symbolic methods

Symbolic systems use explicit rules or knowledge trees defined by experts. They are fully transparent but have limited ability to generalize. For example, Khazaei et al. used fuzzy logic and rule-based systems to predict CVSS scores but could not handle diverse text or capture complete vectors [18]. Elbaz et al. applied linear regression on Bag of Words features, offering interpretability but missing deeper semantic relations [19].

Recent work combines symbolic reasoning with neural models to gain both interpretability and accuracy [20–23]. In

these neuro-symbolic systems, neural layers learn language patterns while symbolic components encode domain logic. In the security domain, recent studies have explored the integration of heterogeneous information sources for vulnerability analysis. Tian et al. [24] combined code semantics with LLM-generated explanations through cross-modal attention, while Zhequ et al. [25] investigated semantic dependency analysis and fusion paths for modeling relationships among code components. These studies illustrate the potential value of multi-source fusion for context-aware vulnerability analysis without implying that a particular fusion mechanism is universally superior. The model *(WSymBERT)* is guided by expert-defined mappings or ontologies that connect key phrases to CVSS metric values. For example, a phrase like "requires local access" can map directly to *Attack Vector = Local*, and "no user interaction required" can map to *User Interaction = None*. When these symbolic cues are incorporated into representation pooling, the model can assign greater emphasis to tokens associated with CVSS-relevant concepts. This design combines contextual language representations with structured symbolic guidance while keeping attribution analysis separate from the pooling mechanism.

### 2.4 Use of external and OSINT data

Some researchers explored the use of Open Source Intelligence (OSINT) data such as CWE identifiers, exploit availability, or attack references to improve prediction accuracy [11]. Kühn et al. found that adding more descriptive text from OSINT sources increased text length but did not always improve results. Their analysis showed that beyond a certain point, additional context leads to "information saturation".

These results suggest that model performance depends more on the quality and domain relevance of information than on text quantity. This observation motivates the use of symbolic guidance to emphasize domain-relevant cues within the pooled representation rather than simply increasing input length.

### 2.5 Explainable AI in vulnerability scoring

As neural models became more accurate, researchers began to study explainability in automated CVSS prediction. Post-hoc methods such as Integrated Gradients, SHAP, and LIME have been used to visualize which words influenced model predictions [26, 27]. Metrics like Infidelity, Sensitivity, and Deletion-AUC were proposed to evaluate the quality of explanations [28]. A recent systematic taxonomy of more than 100 studies on Generative AI in vulnerability assessment and risk management identifies explainability, security, and trustworthiness as recurring challenges across the vulnerability-management lifecycle [29]. This broader perspective motivates evaluating automated scoring systems beyond predictive accuracy alone. Integrated Gradients has also been applied to token- and feature-level attribution in recent NLP-oriented studies [30], while recent work has explored alternative neural-interpretation approaches and benchmarking frameworks for post-hoc NLP explainability [31, 32]. Post-hoc attribution methods characterize model behavior after training but do not themselves inject domain knowledge into the predictive representation. This motivates complementary approaches in which structured security knowledge can guide representation learning while explanation quality is evaluated separately.

Prior research has made strong progress in improving the accuracy and understanding of CVSS prediction, but several important gaps remain. Neural models can achieve high predictive performance, yet their internal reasoning is often not transparent or aligned with CVSS logic. Many systems depend on surface-level word associations rather than learning deeper semantic meaning from vulnerability descriptions. In addition, most existing approaches do not integrate expert knowledge or symbolic reasoning directly into the model, which limits their ability to reflect domain logic. Finally, explainability in prior studies is usually added after training through post-hoc analysis tools, which only describe model behavior instead of shaping it. These gaps highlight the need for an approach that combines neural learning with symbolic guidance to achieve both accuracy and interpretability in automated CVSS prediction.

However, feature-attribution methods may still produce unstable or misleading importance patterns, particularly when interpreted without quantitative validation. Therefore, qualitative attribution analysis should be complemented by quantitative explanation metrics, while symbolic guidance should be interpreted as knowledge injection rather than as direct evidence of model reasoning.

## 3 Proposed method

We propose WSymBERT, a weighted neuro-symbolic framework for predicting the eight CVSS v3.1 Base metrics from vulnerability descriptions. Each metric-specific model combines contextual representations from a fine-tuned SBERT encoder with symbolic guidance derived from expert lexicons and KeyBERT keywords. These sources are integrated through symbolic-guided weighted pooling before metric-specific classification (Fig. 1).

### 3.1 Inputs and notation

Let $d$ be tokenized into $\mathbf{x} = [t_1, \ldots, t_n]$, and let SBERT return contextual token embeddings $H = [h_1, \ldots, h_n]$, with $h_i \in \mathbb{R}^d$. For each token $t_i$, the model computes a learned token-level pooling weight $\alpha_i$. A symbolic weighting factor $w_i$, derived from expert and KeyBERT evidence, is combined

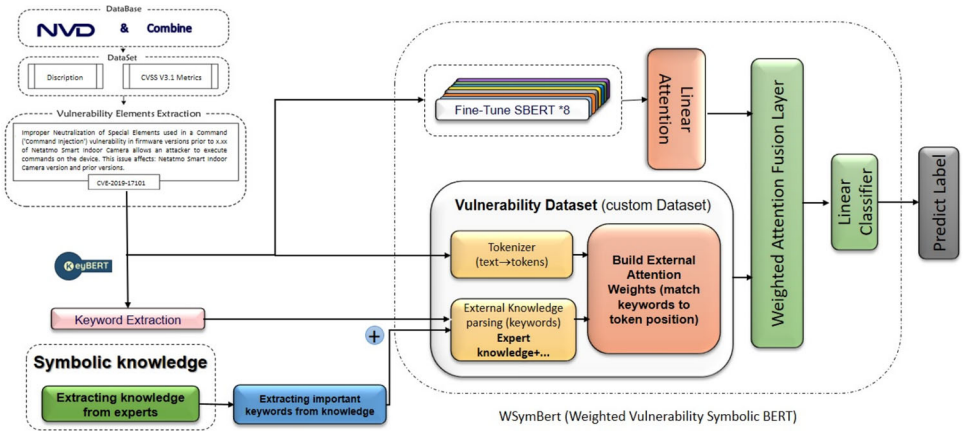



**Fig. 1** System overview of WSymBERT. Eight independent metric-specific models are trained, one for each CVSS Base metric. Each model combines an SBERT encoder with expert- and model-derived symbolic weighting, followed by weighted pooling and a metric-specific classifier

with $\alpha_i$ to obtain the normalized fused pooling weight $\beta_i$. The weighted representation $z_m \in \mathbb{R}^d$ is then constructed for the metric-specific model $m$. A separate classifier predicts the corresponding CVSS metric label from $z_m$.

## 3.2 Symbolic assets

Figure 2 provides an example to show the style of the expert asset used to build lexicons. The decision tree systematically maps common descriptive phrases to the Attack Vector classes: *Network*, *Adjacent*, *Local*, or *Physical*. The resulting leaf sets seed the initial AV lexicon, grounding the symbolic weights in structural domain knowledge.

*Decision trees.* For each Base metric we build a question driven tree from the FIRST scoring guide and security ontologies [33–36]. Paths link common phrases to metric values. For example, "requires local access" maps to *AV = Local*.

*Guiding lexicons.* From the trees we derive per label lexicons of single word cues. Experts review the lists.

*Model keywords.* We extract up to five keywords from each $d$ using KeyBERT [37]. The score for a word $w$ is cosine similarity in the same SBERT space,

$$S(w) = \frac{\langle e_d,\, e_w \rangle}{\|e_d\|\,\|e_w\|}, \tag{1}$$

where $e_d$ is the sentence embedding and $e_w$ is the word embedding.

KeyBERT was applied using the study-specific embedding checkpoint without additional fine-tuning during keyword extraction. Unigram candidates were used with `keyphrase_ngram_range=(1,1)`, no explicit stopword filtering, and `top_n=5`. MMR and diversity-based re-ranking were not applied, and the selected keywords were subsequently treated with equal weights.

## 3.3 Symbolic knowledge extraction

The decision logic of cybersecurity experts in CVSS v3.1 scoring was formalized as traceable, question-driven decision trees, modeling the "question–response–label" reasoning typical in expert assessments. Separate trees were built for the eight Base metrics (AV, AC, PR, UI, S, C, I, A). These trees serve as upstream knowledge-construction resources from which the metric-specific expert lexicons are derived; the resulting lexicons are subsequently used in the symbolic-guided weighted pooling mechanism.

The initial framework was derived from official CVSS guidelines and combined with structured rubrics and security ontologies to create executable question-based trees.

A stratified analysis of 2,000 CVE descriptions aligned these trees with domain-specific NVD text records. This process achieved 83.6% coverage, meaning that 83.6% of these CVE descriptions contained at least one token matched by the trees. The remaining cases without symbolic matches relied purely on the underlying language model semantics. All trees were developed solely on the training set and frozen before evaluation.

To handle multi-word expressions such as "adjacent network," the lexicons include bigrams and n-grams alongside single tokens. When these phrases are identified during preprocessing, symbolic weights are assigned to their constituent parts. Contextual SBERT representations encode information from the surrounding linguistic context and can therefore reduce reliance on isolated lexical matches. However, robustness to negation and other compositional linguistic phenomena was not evaluated separately in the present study.

## 3.4 Symbolic-guided weighted pooling

*Learned pooling weights* A learned pooling mechanism assigns a scalar score to each token and normalizes the scores

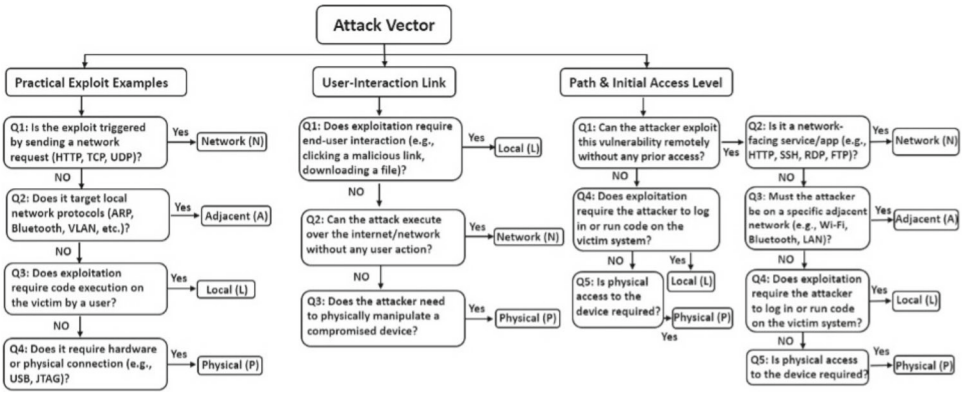



**Fig. 2** Attack Vector decision tree. Phrases map to *Network*, *Adjacent*, *Local*, or *Physical*. The leaf sets seed the AV lexicon

across the sequence:

$$s_i = W_{\text{pool}} h_i + b_{\text{pool}}, \quad \alpha_i = \frac{\exp(s_i)}{\sum_{j=1}^{n} \exp(s_j)}, \quad W_{\text{pool}} \in \mathbb{R}^{1\times d}. \tag{2}$$

*Symbolic weighting factors* Let $\mathcal{K}_{\text{exp}}$ be the expert lexicon for the current metric and $\mathcal{K}_{\text{mdl}}$ be the KeyBERT keywords for $d$. We assign

$$w_i' = \begin{cases} 10, & t_i \in \mathcal{K}_{\text{exp}} \cap \mathcal{K}_{\text{mdl}}, \\ 1, & t_i \in \mathcal{K}_{\text{exp}} \triangle \mathcal{K}_{\text{mdl}}, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

To preserve the neural representation when no symbolic cue is matched, the final weighting factor is defined as $w_i = w_i' + 1$. Consequently, when $w_i' = 0$ for all tokens, the fused weighting reduces, up to the numerical stabilization term, to the learned neural pooling weights $\alpha_i$. The weighting scheme represents three levels of symbolic evidence: no symbolic match ($w_i' = 0$), single-source support ($w_i' = 1$), and joint expert–KeyBERT support ($w_i' = 10$). The larger value encodes stronger dual-source agreement before normalization. This configuration was kept fixed across the reported experiments; a systematic sensitivity analysis of alternative weighting values was not conducted and is treated as a limitation.

*Normalized fused pooling* We form the normalized fused pooling weights and the weighted pooled representation as

$$\beta_i = \frac{\alpha_i\, w_i}{\sum_{j=1}^{n} \alpha_j\, w_j + \varepsilon}, \qquad z_m = \sum_{i=1}^{n} \beta_i\, h_i, \tag{4}$$

with a small $\varepsilon$ for numeric safety. This pooling focuses on tokens that are important both semantically and symbolically.

Here, $\alpha_i$ denotes the learned token-level pooling weight, $w_i$ the symbolic weighting factor, and $\beta_i$ the normalized fused pooling weight used to construct $z_m$. The symbolic-guided weighting mechanism is used for knowledge injection rather than as a standalone explanation method; its token weights are not interpreted as causal evidence of model reasoning, and explanation quality is evaluated separately using attribution-based analyses.

### 3.5 Metric-specific independent prediction

WSymBERT employs eight independently trained models, one for each CVSS Base metric $m \in \{AV, AC, PR, UI, S, C, I, A\}$. Each model contains its own SBERT encoder, symbolic-guided weighted pooling mechanism, and metric-specific classifier. No encoder parameters are shared across metrics. For metric $m$ with label set $\mathcal{Y}_m$, the classifier is defined as

$$\hat{p}_m = \text{softmax}(W_m z_m + b_m), \qquad \hat{y}_m = \arg\max_{y\in\mathcal{Y}_m} \hat{p}_m(y). \tag{5}$$

Each metric-specific model is optimized independently using only the labels of its corresponding CVSS metric:

$$\mathcal{L}_m = \text{CE}\big(\hat{p}_m, y_m\big). \tag{6}$$

No combined multi-metric loss is used (Fig. 3).

### 3.6 Training algorithm

The following procedure trains one metric-specific WSymBERT model. The same procedure is executed independently for each of the eight CVSS Base metrics.

KeyBERT keywords are cached before model optimization and remain fixed during the training of each metric-specific model.

### 3.7 Explainability

Integrated Gradients is used to attribute token contributions for each metric-specific model [38]. Because saliency meth-

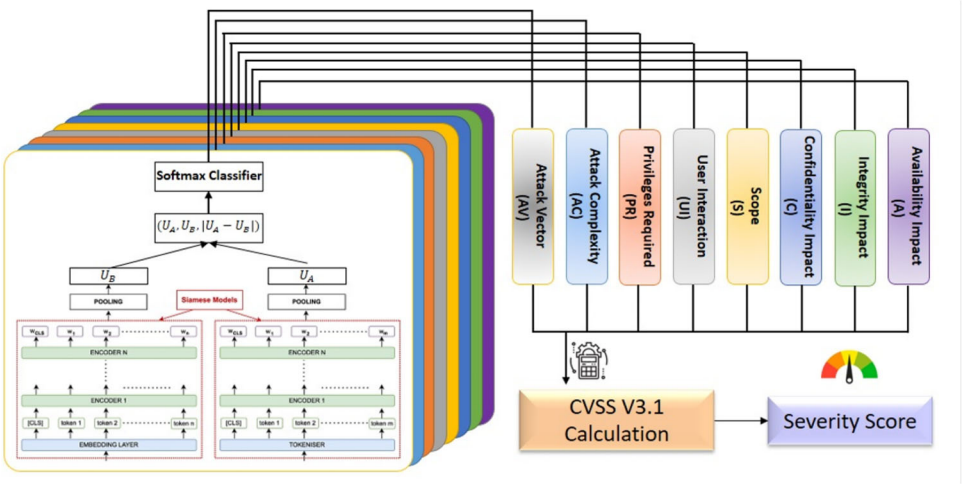



**Fig. 3** Overall metric-specific prediction architecture. Eight independently trained WSymBERT models generate the eight CVSS v3.1 Base metrics (AV, AC, PR, UI, S, C, I, and A), which are subsequently provided to the standard CVSS v3.1 calculator to compute the final severity score

**Algorithm 1** Independent training of one metric-specific WSymBERT model

**Require:** Metric $m$, SBERT encoder, expert lexicon $\mathcal{K}_{\text{exp}}^{(m)}$, training data $D_m$, epochs $E$

Pre-compute and cache KeyBERT keywords $\mathcal{K}_{\text{mdl}}$ for each description using Eq. (1)
**for** epoch $= 1$ to $E$ **do**
  **for** each batch of descriptions $d$ and metric labels $y_m$ in $D_m$ **do**
    tokenize $d$ and encode contextual representations $H = [h_1, \ldots, h_n]$
    compute learned pooling weights $\alpha_i$ using Eq. (2)
    construct symbolic weighting factors $w_i$ using Eq. (3)
    compute fused pooling weights $\beta_i$ and representation $z_m$ using Eq. (4)
    compute $\hat{p}_m$ using Eq. (5)
    update only the metric-specific model parameters by minimizing $\mathcal{L}_m$ in Eq. (6)
  **end for**
**end for**

ods can fail model- or data-randomization sanity checks, attribution maps are not treated as self-validating explanations [39]. Explanation quality is evaluated separately from the pooling mechanism using attribution- and perturbation-based measures under explicitly defined reference configurations (Fig. 4).

*Remarks.* The independent metric-specific design enables component-level training and error analysis while leaving the standard downstream CVSS v3.1 score computation unchanged.

## 4 Experiments

We evaluated WSymBERT against SBERT [9], DistilBERT [11], and three domain-specific vulnerability-scoring baselines: CVSS-BERT [10], SecureBERT, and CVEDrill [8]. WSymBERT training used an NVIDIA GeForce RTX 2080 GPU, AdamW with a learning rate of $2 \times 10^{-5}$, batch size 16,

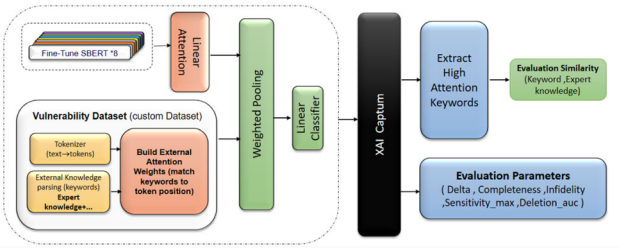



**Fig. 4** Explainability flow. Model outputs, IG attributions, and expert keyword overlap give both visual and numeric checks

15 epochs, and CrossEntropyLoss without class weighting. The best checkpoint for each metric-specific WSymBERT model was selected using validation accuracy, while the test partition was reserved for final evaluation. For the added re-evaluations, CVSS-BERT used the BERT-small configuration with three frozen-encoder epochs followed by three fine-tuning epochs (batch size 16), whereas SecureBERT and CVEDrill were trained for 10 epochs with a learning rate of $10^{-5}$ and batch size 8. The supplied baseline runs used a maximum sequence length of 128 and seed 42. The experimental comparison focuses on predictive effectiveness rather than computational efficiency. Parameter counts and wall-clock GPU training times were not systematically recorded; therefore, no claim of computational-cost superiority is made.

### 4.1 Datasets

Two datasets were used for fine-tuning and evaluation: the *Combine dataset* and the *NVD dataset*. Both contain labeled CVE entries with textual descriptions and corresponding CVSS v3.1 base vector metrics, but they differ in scope, source, and data quality.

*Combine Dataset* The Combine dataset is a merged benchmark introduced in earlier work [11], designed for reproducible CVSS prediction research. It integrates vul-

nerability records from multiple public sources, including the National Vulnerability Database (NVD), ExploitDB, and other vendor repositories. The dataset covers the years 2016–2021 and contains approximately 101,735 entries. Each record includes a natural-language description of the vulnerability, its associated CVSS v3 base vector (AV, AC, PR, UI, S, C, I, A), and the corresponding numerical base score. Because it combines data from heterogeneous sources, the dataset provides richer linguistic diversity and terminology coverage, though it also contains occasional inconsistencies and overlapping records.

*NVD Dataset* The NVD dataset used in this study was fully collected, parsed, and cleaned by the authors from the official National Vulnerability Database.[1] It includes 153,106 records spanning 2016–2025. The records used for modeling contain complete CVSS v3.1 Base-metric labels together with their vulnerability descriptions and associated CVSS information. Compared to the Combine dataset, this corpus is more consistent and structured but includes shorter text fields, which can make semantic inference more challenging for language models.

*Data Splitting and Deduplication* Strict deduplication was applied within each dataset before splitting. Each corpus was then partitioned into stratified 80%/10%/10% training, validation, and test sets. Time-aware splitting was not used; therefore, the primary evaluation measures in-distribution predictive performance and does not establish temporal generalization to future vulnerabilities.

Cross-corpus transfer was evaluated separately by applying each source-trained model to a randomly selected subset of the other corpus without additional training or fine-tuning. Specifically, Combine-trained models were evaluated on NVD samples, while NVD-trained models were evaluated on Combine samples.

## 4.2 Evaluation results

Tables 2 and 3 present the performance results across both datasets and all CVSS Base metrics.

For each of the eight CVSS Base metrics, Accuracy, macro-Precision, macro-Recall, and macro-F1 are reported. Tables 2 and 3 contain single-run point estimates on held-out test partitions. Run-to-run variability across multiple random seeds, confidence intervals, and paired significance tests were not evaluated; therefore, the reported differences are interpreted as observed empirical performance rather than formal evidence of statistical significance or run-to-run stability.

Table 3 additionally includes the supplied re-evaluations of CVSS-BERT, SecureBERT, and CVEDrill. The supplied baseline files use a stratified 80/10/10 split with seed 42 and report Accuracy together with macro-Precision, macro-Recall, and macro-F1. Because the original WSymBERT split identifiers were not preserved, the added results are interpreted as aligned point-estimate comparisons rather than sample-wise paired evaluations.

**Table 2** Comparison of model performance on the combine dataset

| Model | Metric | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| DistilBERT | AV | 91.04 | 82.83 | 75.01 | 78.11 |
| | AC | 92.69 | 96.34 | 57.00 | 56.10 |
| | PR | 84.02 | 76.47 | 77.50 | 76.84 |
| | UI | 91.19 | 90.49 | 90.29 | 90.39 |
| | S | 95.69 | 93.50 | 90.97 | 92.17 |
| | C | 86.28 | 83.98 | 84.17 | 84.07 |
| | I | 87.25 | 87.13 | 85.75 | 86.40 |
| | A | 88.45 | 79.72 | 73.22 | 75.74 |
| SBERT | AV | 93.26 | 85.77 | 81.69 | 83.62 |
| | AC | 96.13 | 89.55 | 79.56 | 83.70 |
| | PR | 87.62 | 84.15 | 81.41 | 82.69 |
| | UI | 94.53 | 94.11 | 93.99 | 94.05 |
| | S | 96.74 | 94.97 | 93.36 | 94.14 |
| | C | 89.47 | 88.18 | 87.14 | 87.63 |
| | I | 89.65 | 89.42 | 88.74 | 89.02 |
| | A | 90.93 | 86.11 | 76.44 | 80.05 |
| **WSymBERT** | AV | **95.73** | **95.70** | **95.73** | **95.71** |
| | AC | **97.83** | **97.73** | **97.83** | **97.76** |
| | PR | **90.38** | **90.31** | **90.38** | **90.33** |
| | UI | **96.14** | **96.14** | **96.14** | **96.14** |
| | S | **97.64** | **97.63** | **97.64** | **97.63** |
| | C | **93.74** | **93.76** | **93.74** | **93.72** |
| | I | **92.24** | **92.25** | **92.24** | **92.24** |
| | A | **93.70** | **93.58** | **93.70** | **93.58** |

Best values per metric are in bold

On the Combine dataset, WSymBERT achieved a mean accuracy of 94.7% across the eight CVSS Base metrics, while the corresponding mean on the NVD dataset was 96.2%. In the expanded Table 3, WSymBERT retained the highest observed Accuracy, macro-Precision, macro-Recall, and macro-F1 values for every reported CVSS metric. The mean NVD accuracy was 96.2% for WSymBERT, compared with 91.7% for SecureBERT, 91.6% for SBERT, 91.6% for CVSS-BERT, 91.4% for CVEDrill, and 89.4% for DistilBERT. These values are descriptive point estimates and are not interpreted as paired statistical evidence. The largest observed gains for the complete WSymBERT configuration occur on several difficult or imbalanced metrics, including Privileges Required and Availability. Because no component-wise ablation or paired significance test was performed, these differences are not attributed causally to any single symbolic component.

[1] https://nvd.nist.gov/.

**Table 3** Comparison of model performance on the author-collected NVD dataset

| Model | Metric | Acc | Prec | Rec | F1 |
|---|---|---|---|---|---|
| DistilBERT | AV | 91.59 | 81.65 | 75.36 | 77.89 |
| | AC | 94.47 | 97.24 | 55.00 | 58.33 |
| | PR | 82.29 | 77.81 | 75.32 | 76.47 |
| | UI | 92.08 | 91.52 | 90.86 | 91.18 |
| | S | 96.02 | 95.44 | 90.83 | 92.93 |
| | C | 85.18 | 84.54 | 81.83 | 83.05 |
| | I | 86.05 | 85.67 | 85.26 | 85.46 |
| | A | 87.49 | 74.95 | 70.03 | 71.97 |
| SBERT | AV | 93.46 | 83.82 | 80.73 | 82.14 |
| | AC | 96.31 | 83.54 | 78.89 | 81.01 |
| | PR | 85.48 | 81.60 | 79.94 | 80.74 |
| | UI | 94.56 | 94.12 | 93.82 | 93.97 |
| | S | 96.46 | 94.42 | 93.52 | 93.96 |
| | C | 87.62 | 86.00 | 86.08 | 86.04 |
| | I | 88.75 | 88.82 | 87.79 | 88.27 |
| | A | 90.23 | 79.18 | 73.02 | 75.39 |
| CVSS-BERT | AV | 92.95 | 89.68 | 75.65 | 81.36 |
| | AC | 95.75 | 81.92 | 78.94 | 80.35 |
| | PR | 85.80 | 82.92 | 79.20 | 80.88 |
| | UI | 94.48 | 94.15 | 93.60 | 93.86 |
| | S | 96.51 | 94.94 | 92.92 | 93.89 |
| | C | 88.60 | 88.37 | 85.72 | 86.93 |
| | I | 88.60 | 89.26 | 86.99 | 87.99 |
| | A | 90.14 | 85.17 | 68.94 | 72.77 |
| SecureBERT | AV | 93.08 | 83.63 | 80.20 | 81.82 |
| | AC | 96.11 | 84.39 | 78.85 | 81.34 |
| | PR | 85.64 | 81.35 | 80.22 | 80.77 |
| | UI | 94.30 | 93.58 | 93.86 | 93.72 |
| | S | 96.15 | 93.56 | 93.20 | 93.38 |
| | C | 88.79 | 88.01 | 86.47 | 87.20 |
| | I | 88.72 | 88.34 | 87.90 | 88.11 |
| | A | 90.65 | 79.45 | 73.43 | 75.80 |
| CVEDrill | AV | 92.83 | 82.38 | 81.85 | 82.10 |
| | AC | 95.55 | 80.29 | 80.70 | 80.49 |
| | PR | 85.19 | 80.88 | 79.15 | 79.98 |
| | UI | 94.59 | 94.17 | 93.84 | 94.00 |
| | S | 96.66 | 95.49 | 92.88 | 94.12 |
| | C | 88.37 | 87.24 | 86.56 | 86.89 |
| | I | 88.33 | 87.57 | 87.81 | 87.68 |
| | A | 90.05 | 74.76 | 75.58 | 75.11 |
| **WSymBERT** | AV | **97.22** | **97.22** | **97.22** | **97.21** |
| | AC | **96.47** | **96.18** | **96.47** | **96.02** |
| | PR | **94.23** | **94.20** | **94.23** | **94.18** |
| | UI | **97.44** | **97.44** | **97.44** | **97.44** |
| | S | **98.79** | **98.78** | **98.79** | **98.78** |
| | C | **95.12** | **95.11** | **95.12** | **95.09** |
| | I | **95.09** | **95.10** | **95.09** | **95.08** |
| | A | **95.46** | **95.38** | **95.46** | **95.38** |

Best observed values per metric are shown in bold

**Table 4** Percentage of each label for every CVSS base metric in the NVD dataset

| Metric | Label distribution (%) |
|---|---|
| Attack vector (AV) | Network: 73.9, Local: 23.0, Adjacent: 2.1, Physical: 1.0 |
| Attack complexity (AC) | Low: **94.0**, High: **6.0** |
| Privileges required (PR) | None: 62.4, Low: 29.4, High: 8.2 |
| User interaction (UI) | None: 65.4, Required: 34.6 |
| Scope (S) | Unchanged: 82.2, Changed: 17.8 |
| Confidentiality (C) | High: 57.5, None: 21.7, Low: 20.7 |
| Integrity (I) | High: 49.5, None: 31.1, Low: 19.4 |
| Availability (A) | High: 57.5, None: 40.3, Low: 2.2 |

**Table 5** Single-run point estimates of MSE and MAE for CVSS v3.1 Base Score prediction

| Model | MSE | MAE |
|---|---|---|
| DistilBERT | 1.85 | 0.71 |
| WSymBERT | **1.53** | **0.65** |

## 4.3 In-domain dataset evaluation

The in-domain results were obtained independently within the NVD and Combine corpora. Table 4 summarizes the NVD class distributions and shows substantial imbalance for several metrics, particularly Attack Complexity and Availability. Despite this imbalance, the complete WSymBERT configuration achieved the highest observed performance among the evaluated models for the reported NVD metrics.

## 4.4 Evaluation of errors

To evaluate numerical CVSS Base Score prediction, the eight predicted Base-metric labels for each held-out CVE were passed to the standard CVSS v3.1 Base Score calculator after metric prediction. MSE and MAE were then computed between the resulting predicted Base Scores and the corresponding reference Base Scores over the held-out test data. Lower values indicate smaller numerical prediction error (Table 5).

**Table 6** Representative integrated gradients attribution outputs for WSymBERT and DistilBERT across CVSS metrics

| Metric | Description | Top-attribution tokens: WSymBERT | Top-attribution tokens: DistilBERT |
|---|---|---|---|
| AV | IBM AIX 7.1, 7.2, 7.3, and VIOS 3.1 could allow a non-privileged local user to exploit a vulnerability in the invscout command to execute arbitrary commands. IBM X-Force ID: 251207 | ['invscout', 'commands', 'privileged', '251207', 'exploit', 'execute', 'IBM', 'local', 'arbitrary', 'VIOS'] | ['local', 'VIOS', 'AIX', 'could', '1', '1', 'the', '7', 'and', 'co'] |
| AC | An information disclosure vulnerability exists when the Windows kernel improperly handles objects in memory, aka 'Windows Kernel Information Disclosure Vulnerability'. This CVE ID is unique from CVE-2020-1367, CVE-2020-1389, CVE-2020-1419 | ['disclosure', 'Disclosure', 'memory', 'exists', 'information', 'kernel', 'unique', 'objects', 'Vulnerability', 'CVE'] | ['information', '2020', '7', 'aka', 'when', 'in', 'ly', 'CVE', 'E', 'An'] |
| PR | customapp in Cybozu Office 9.9.0 through 10.3.0 allows remote authenticated users to cause a denial of service via unspecified vectors, a different vulnerability than CVE-2015-8489 | ['authenticated', 'unspecified', 'users', 'Cybozu', '8489', 'denial', 'service', 'allows', 'customapp', 'different'] | ['authenticated', 'customapp', '0', 'users', 'denial', 'a', '9', 'of', '8489', 'bozu'] |
| UI | The AJAX Login and Registration modal popup + inline form plugin for WordPress is vulnerable to Reflected Cross-Site Scripting due to the use of add_query_arg without appropriate escaping on the URL in all versions up to, and including, 2.24. This makes it possible for unauthenticated attackers to inject arbitrary web scripts in pages that execute if they can successfully trick a user into performing an action such as clicking on a link | ['Reflected', 'unauthenticated', 'AJAX', 'successfully', 'Scripting', 'performing', 'execute', 'scripts', 'attackers', 'trick'] | ['without', 'pop', 'plug', 'up', 'an', 'in', 'to', 'including', 'web', '2'] |
| S | An arbitrary file write vulnerability in RGCMS v1.06 allows attackers to execute arbitrary code via a crafted PHP file | ['attackers', 'RGCMS', 'PHP', 'vulnerability', 'crafted', 'execute', 'code', 'v1', 'file', 'arbitrary'] | ['to', 'execute', 'RGCMS', 'a', 'file', '06', 'write', 'attackers', 'file', 'An'] |
| C | IBM CICS TX 11.1 could allow an attacker with physical access to the system to execute code due using [sic] a back and refresh attack. IBM X-Force ID: 229312. | ['execute', '229312', 'physical', 'attacker', 'refresh', 'IBM', 'using', 'allow', 'TX', 'could'] | ['code', 'TX', '229312', 'X', 'and', '1', 'to', 'using', 'execute', '11'] |
| I | On BIG-IP 14.0.0−14.1.0.1, 13.0.0−13.1.1.4, 12.1.0−12.1.4, 11.6.1−11.6.3.4, and 11.5.2−11.5.8, users with the Resource Administrator role can modify sensitive portions of the filesystem if provided Advanced Shell Access, such as editing /etc/passwd. This allows modifications to user objects and is contrary to our definition for the Resource Administrator (RA) role restrictions | ['Administrator', 'Administrator', 'users', 'user', 'passwd', 'Resource', 'Advanced', 'filesystem', 'objects', 'modify'] | ['contrary', 'editing', 'modify', '11', 'the', '4', 'our', '0', '3', 'role'] |
| A | The mintToken function of a smart contract implementation for SECoin, an Ethereum token, has an integer overflow that allows the owner of the contract to set the balance of an arbitrary user to any value | ['mintToken', 'Ethereum', 'SECoin', 'implementation', 'value', 'owner', 'balance', 'contract', 'any', 'function'] | ['mintTok', 'the', 'flow', 'the', 'for', 'arbitrary', 'SECoin', 'an', 'allows', 'contract'] |

**Table 7** Comparison of heuristic noise rates across the four reported CVSS Base metrics for WSymBERT and DistilBERT

| CVSS metric | Model | Stopwords (%) | Numeric tokens (%) | Short tokens (%) | Noise rate (%) |
|---|---|---|---|---|---|
| AV | DistilBERT | 18.4 | 7.2 | 5.5 | 31.1 |
| | WSymBERT | 1.2 | 1.5 | 2.0 | **4.7** |
| AC | DistilBERT | 21.5 | 5.8 | 4.1 | 31.4 |
| | WSymBERT | 0.8 | 1.1 | 2.3 | **4.2** |
| PR | DistilBERT | 19.9 | 6.3 | 5.0 | 31.2 |
| | WSymBERT | 1.0 | 1.3 | 2.1 | **4.4** |
| UI | DistilBERT | 17.2 | 8.6 | 6.5 | 32.3 |
| | WSymBERT | 1.4 | 1.6 | 2.0 | **5.0** |

**Table 8** Results of interpretability metrics for the SBERT model on the NVD dataset

| Metric | Baseline | Δ | Completeness | Infidelity | Sensitivity | Deletion AUC |
|---|---|---|---|---|---|---|
| AV | pad | 0.0129 | 6.457 | 0.584 | 2.790 | 0.198 |
| | one | 0.0147 | 6.352 | 11.329 | 1.361 | 0.145 |
| AC | pad | 0.0215 | 3.438 | 0.107 | 2.631 | 0.759 |
| | one | 0.0092 | 2.180 | 7.438 | 1.503 | 0.671 |
| PR | pad | 0.0050 | 5.157 | 0.350 | 3.129 | 0.581 |
| | one | 0.0035 | 3.499 | 12.068 | 1.762 | 0.440 |
| UI | pad | 0.0558 | 4.234 | 0.154 | 2.515 | 0.613 |
| | one | 0.0164 | 3.833 | 5.296 | 1.722 | 0.474 |
| S | pad | 0.0053 | 4.051 | 0.069 | 1.978 | 0.704 |
| | one | 0.0107 | 3.934 | 3.191 | 1.238 | 0.512 |
| C | pad | 0.0100 | 5.776 | 0.333 | 2.645 | 0.255 |
| | one | 0.0354 | 6.005 | 11.372 | 1.416 | 0.217 |
| I | pad | 0.0120 | 5.289 | 0.572 | 2.044 | 0.309 |
| | one | 0.0113 | 5.800 | 7.452 | 1.396 | 0.259 |
| A | pad | 0.0243 | 5.125 | 0.235 | 2.584 | 0.554 |
| | one | 0.0109 | 4.803 | 8.492 | 1.301 | 0.399 |

WSymBERT produced lower observed MSE and MAE than DistilBERT in this evaluation. Because repeated training runs were not performed, these values are not interpreted as estimates of run-to-run stability.

## 4.5 Integrated gradients analysis

Integrated Gradients was used to identify influential tokens for each CVSS metric. The examples in Table 6 indicate that WSymBERT assigns greater attribution to security-relevant terms that overlap with the expert-derived vocabulary than the evaluated DistilBERT baseline. These attribution patterns describe model behavior under the selected explanation method and are not interpreted as causal evidence of human-like or expert-equivalent reasoning.

## 4.6 Noise analysis

Not all tokens highlighted by interpretability methods provide meaningful or domain-relevant information. To measure this, we defined four related metrics that capture different types of *semantic noise* among the top-ranked tokens (Top-$k$):

- **Stopwords:** Common words that appear very often but carry little meaning, such as *the*, *a*, *of*, *to*, and *in*. These words do not help the model distinguish between vulnerability classes and may lower interpretability quality.
- **Numeric or Version Tokens:** Tokens made up only of numbers or software version patterns, such as *123* or *7.5.1*. These tokens vary widely and usually do not have a consistent semantic role.
- **Short Tokens:** Tokens with two or fewer characters, such as *ip*, *os*, or *c*. These are often ambiguous and may overlap by chance, providing little useful information.
- **Noise Rate:** The proportion of tokens that belong to any of the three categories above among the total number of top-$k$ tokens. This value gives an overall measure of the amount of noise in the model's explanations.

**Table 9** Interpretability metrics for WSymBERT on the NVD dataset across the four Integrated Gradients reference configurations

| Metric | Reference | **Mean $\|\Delta\|$** | Mean \|Comp. residual\| | Infidelity | Sensitivity | Deletion AUC |
|---|---|---|---|---|---|---|
| AV | $B_1$ | 0.1272 | 0.3229 | 0.00039 | 0.5849 | 0.3209 |
| | $B_2$ | 0.1288 | 0.3234 | 0.00040 | 0.5569 | 0.3124 |
| | $B_3$ | 0.0177 | 0.2980 | 0.01523 | 0.3026 | 0.2974 |
| | $B_4$ | 0.0175 | 0.2994 | 0.01484 | 0.2993 | 0.2908 |
| AC | $B_1$ | 0.5424 | 0.9856 | 0.00096 | 1.3728 | 0.3742 |
| | $B_2$ | 0.5403 | 0.9841 | 0.00098 | 1.3287 | 0.3760 |
| | $B_3$ | 0.1616 | 0.6928 | 0.06376 | 0.4407 | 0.3667 |
| | $B_4$ | 0.1335 | 0.6889 | 0.06542 | 0.4231 | 0.3589 |
| PR | $B_1$ | 0.0008 | 0.7312 | 0.00071 | 0.3549 | 0.2940 |
| | $B_2$ | 0.0008 | 0.7326 | 0.00070 | 0.3578 | 0.2833 |
| | $B_3$ | 0.0040 | 0.6627 | 0.02940 | 0.3263 | 0.2802 |
| | $B_4$ | 0.0041 | 0.6740 | 0.03011 | 0.3256 | 0.2597 |
| UI | $B_1$ | 0.1501 | 0.3785 | 0.00041 | 0.7146 | 0.3116 |
| | $B_2$ | 0.1514 | 0.3780 | 0.00042 | 0.7103 | 0.3039 |
| | $B_3$ | 0.0290 | 0.2161 | 0.01156 | 0.4217 | 0.2857 |
| | $B_4$ | 0.0296 | 0.2221 | 0.01053 | 0.4212 | 0.2846 |
| S | $B_1$ | 0.0847 | 0.8914 | 0.00070 | 0.6332 | 0.3439 |
| | $B_2$ | 0.0826 | 0.8930 | 0.00068 | 0.5664 | 0.3406 |
| | $B_3$ | 0.0721 | 0.4754 | 0.02328 | 0.5374 | 0.3220 |
| | $B_4$ | 0.0736 | 0.4342 | 0.02462 | 0.5395 | 0.3160 |
| C | $B_1$ | 0.0652 | 0.4795 | 0.00105 | 0.4356 | 0.2477 |
| | $B_2$ | 0.0654 | 0.4801 | 0.00102 | 0.4374 | 0.2416 |
| | $B_3$ | 0.0310 | 0.6287 | 0.04687 | 0.2985 | 0.2286 |
| | $B_4$ | 0.0314 | 0.6412 | 0.05101 | 0.2967 | 0.2175 |
| I | $B_1$ | 0.0922 | 0.7865 | 0.00173 | 0.4453 | 0.3734 |
| | $B_2$ | 0.0925 | 0.7869 | 0.00167 | 0.4253 | 0.3589 |
| | $B_3$ | 0.0317 | 0.8075 | 0.05090 | 0.2830 | 0.3406 |
| | $B_4$ | 0.0303 | 0.8086 | 0.04924 | 0.2823 | 0.3230 |
| A | $B_1$ | 0.0648 | 1.0157 | 0.00032 | 0.4675 | 0.3897 |
| | $B_2$ | 0.0652 | 1.0174 | 0.00034 | 0.4589 | 0.3933 |
| | $B_3$ | 0.1259 | 0.5334 | 0.01002 | 0.4324 | 0.3730 |
| | $B_4$ | 0.1289 | 0.5349 | 0.01034 | 0.4206 | 0.3703 |

Values are means over 1,034 samples per configuration; $|\Delta|$ and the completeness residual are reported as mean absolute values
*Reference configurations:* $B_1$ = Pad-Embedding + Mask-Weight; $B_2$ = Pad-Embedding + One-Weight; $B_3$ = One-Embedding + Mask-Weight; $B_4$ = One-Embedding + One-Weight

For each of the four CVSS Base metrics reported in Table 7, the top-10 tokens were selected according to their Integrated Gradients (IG) attribution scores, and the noise metrics were computed for both WSymBERT and DistilBERT. Representative examples and values are shown in Table 7.

Under the defined noise heuristic, WSymBERT produced lower observed noise rates than DistilBERT across the reported components. This indicates cleaner attribution patterns under the present evaluation criteria, but it does not by itself establish causal reasoning, human-equivalent interpretability, or deployment trustworthiness.

## 4.7 Interpretability evaluation

We evaluate attribution behavior using Integrated Gradients and perturbation-based metrics under explicitly defined reference configurations. The reported values are descriptive point estimates under the stated representation, reference, and perturbation protocol; their raw magnitudes are not interpreted as architecture-independent evidence of explanation superiority (Table 9).

$B_1$–$B_4$ denote Integrated Gradients reference configurations rather than predictive baseline models. For token embeddings, the two references are the embeddings of an all-padding-token sequence and an all-one tensor with the same shape as the input embeddings. For the weighting input, the standard binary attention mask and an all-one vector are used as references. Their Cartesian combinations define $B_1$ (Pad-Embedding + Mask-Weight), $B_2$ (Pad-Embedding + One-Weight), $B_3$ (One-Embedding + Mask-Weight), and $B_4$ (One-Embedding + One-Weight).

The attribution metrics vary across CVSS metrics and reference configurations. Accordingly, no single metric is used as standalone evidence of explanation quality, and raw values are interpreted only within the specified evaluation protocol. In particular, $\Delta$, Infidelity, Sensitivity, and Deletion AUC depend on the selected reference, representation, output function, and perturbation procedure. Cross-model comparisons are therefore reported descriptively rather than as architecture-independent absolute measures.

Some WSymBERT reference configurations, particularly AC under $B_1$ and $B_2$, yield larger mean $|\Delta|$ values than the SBERT $\Delta$ values reported in Table 8. Because the reference constructions and aggregation procedures are not identical across the two tables, these raw magnitudes are not treated as directly comparable. Accordingly, $|\Delta|$ is reported descriptively and is not used alone as evidence of explanation quality.

## 5 Conclusion and future work

This study presented WSymBERT, a neuro-symbolic framework composed of eight independently trained metric-specific models for predicting the CVSS v3.1 Base metrics from vulnerability descriptions. Each model combines an SBERT encoder with symbolic-guided weighted pooling derived from expert and KeyBERT evidence. WSymBERT achieved mean accuracies of 96.2% on NVD and 94.7% on Combine across the eight Base metrics and obtained the highest observed performance among the evaluated models in the reported in-domain comparisons. Attribution-based analyses further showed greater emphasis on security-relevant terms under the specified explanation protocol. These findings support the potential of explicit domain guidance for automated CVSS prediction, while the statistical, temporal, computational, and explainability limitations described below should be considered when interpreting the results.

### 5.1 Limitations

Several limitations should be considered when interpreting the results. The primary evaluations use random stratified splits rather than time-aware partitions; therefore, temporal generalization to future vulnerabilities is not established. Cross-corpus transfer is limited to the NVD and Combine corpora and does not establish unrestricted domain generalization. The predictive results are single-run point estimates without multi-seed uncertainty estimates or paired significance tests. WSymBERT also depends on fixed CVSS v3.1 expert-derived resources and predefined symbolic weighting; component-wise ablation, systematic weight sensitivity, robustness to incomplete or evolving lexicons, and transfer to other CVSS versions were not evaluated. The long-term maintenance cost of the expert resources and the computational cost of training eight independent metric-specific models were not systematically benchmarked. Finally, attribution results depend on representation, reference, output, and perturbation choices and should not be interpreted as architecture-independent or causal evidence of model reasoning. Robustness to deliberately crafted descriptions designed to activate symbolic cues was also not experimentally evaluated.

### 5.2 Future work

Future work should prioritize time-aware evaluation, broader cross-domain testing, multi-seed statistical analysis, component-level ablation, symbolic-weight sensitivity, computational-cost benchmarking, and systematic evaluation of expert-resource maintenance. Additional extensions include structured sources such as CWE, CPE, and Exploit-DB, dynamic CVSS metrics, cybersecurity knowledge graphs, confidence calibration, adversarial-cue testing, and human-in-the-loop deployment.

**Supplementary Information** The online version contains supplementary material available at https://doi.org/10.1007/s10207-026-01342-x.

**Acknowledgements** This work was supported by: The SERICS project (PE00000014) under the NRRP MUR program (funded by the European Union - NextGenerationEU), supporting A. Majd, S. L. Mirtaheri, and A. Pugliese. The ENFIELD (European Lighthouse on Safe and Trustworthy AI) project, funded by the European Union's Horizon Europe research and innovation programme under Grant Agreement No. 101120658, supporting A. Akbarzadeh, S. L. Mirtaheri, and G. Spathoulas.

**Author Contributions** S.L.M. conceptualized the study, developed the methodology, and wrote the main manuscript text. A.M. developed the software, conducted the machine learning experiments, and prepared the figures and tables. R.S. and A.A. contributed to data curation, model validation, and the explainability analysis. G.S. provided domain expertise in cybersecurity and contributed to the interpretation of the results. A.P. supervised the research project and acquired funding. All authors reviewed the manuscript.

**Funding** Open access funding provided by Università della Calabria within the CRUI-CARE Agreement.

**Data Availability** The datasets generated and analyzed during the current study for the experiments are available in the following: https://github.com/ShahbazianR/WSymBert.git

## Declarations

**Conflict of interest** The authors declare that they have no conflict of interest.

**Use of AI and AI-assisted technologies** During the preparation of this work, the authors used GPT-4 and Quillbot in order to improve the linguistic quality, formatting of LaTeX tables, and overall readability of the manuscript. After using this tool, the authors reviewed and edited the content as needed and took full responsibility for the content of the publication.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

## References

1. Yin, J., Hong, W., Wang, H., Cao, J., Miao, Y., Zhang, Y.: A compact vulnerability knowledge graph for risk assessment. ACM Trans. Knowl. Discov. Data **18**(8), 1–17 (2024)
2. Ouraou, M.: Beyond the cvss: rethinking the contextualisation of cves in a connected world. In: European Conference on Cyber Warfare and Security, vol. 24(1), pp. 490–500 (2025)
3. FIRST.Org. Inc. Common vulnerability scoring system v3.1: Specification document. https://www.first.org/cvss/v3-1/specification-document, 2019. Version 3.1
4. Spanos, G., Angelis, L.: A multi-target approach to estimate software vulnerability characteristics and severity scores. J. Syst. Softw. **146**, 152–166 (2018)
5. Wang, P., Zhou, Y., Sun, B., Zhang, W.: Intelligent prediction of vulnerability severity level based on text mining and xgboost. In: International Conference on Advanced Computational Intelligence (ICACI), pp. 72–77. IEEE (2019)
6. Chen, J., Kudjo, P.K., Mensah, S., Brown, S.A., Akorfu, G.: An automatic software vulnerability classification framework using term frequency-inverse gravity moment and feature selection. J. Syst. Softw. **167**, 110616 (2020)
7. Mirtaheri, S.L., Pugliese, A., Movahed, N., Shahbazian, R.: A comparative analysis on using gpt and bert for automated vulnerability scoring. Intell. Syst. Appl. **26**, 200515 (2025)
8. Aghaei, E., Al-Shaer, E., Shadid, W., Niu, X.: Automated cve analysis for threat prioritization and impact prediction (2023) arXiv preprint. arXiv:1705.07874
9. Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using Siamese Bert-networks (2019) arXiv preprint. arXiv:1908.10084
10. Shahid, M.R., Debar, H.: Cvss-bert: explainable natural language processing to determine the severity of a computer security vulnerability from its description. In: IEEE International Conference on Machine Learning and Applications (ICMLA), pp. 1600–1607. IEEE (2021)
11. Kühn, P., Relke, D.N., Reuter, C.: Common vulnerability scoring system prediction based on open source intelligence information sources. Comput. Secur. **131**, 103286 (2023)
12. Mirtaheri, S.L., Pugliese, A.: Leveraging generative ai to enhance automated vulnerability scoring. In: 2024 IEEE Conference on Dependable, Autonomic and Secure Computing (DASC), pp. 57–64 (2024)
13. Mirtaheri, S.L., Pugliese, A., Movahedkor, N., Majd, A.: Advanced automated vulnerability scoring: Improving performance with a fine-tuned bert-cnn model. In: 2024 11th International Symposium on Telecommunications (IST), pp. 109–113 (2024)
14. Mirtaheri, S.L., Pugliese, A., Pascucci, V.: Automated vulnerability score prediction through lightweight generative ai. Knowl.-Based Syst. **329**, 114406 (2025)
15. Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R.S., Brendel, W., Bethge, M., Wichmann, F.A.: Shortcut learning in deep neural networks. Nat. Mach. Intell. **2**(11), 665–673 (2020)
16. Gururangan, S., Swayamdipta, S., Levy, O., Schwartz, R., Bowman, S.R., Smith, N.A.: Annotation artifacts in natural language inference data. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol. 2 (Short Papers), pp. 107–112 (2018)
17. Jain, S., Wallace, B.: Attention is not explanation (2019). arXiv preprint arXiv:1705.07874
18. Khazaei, A., Ghasemzadeh, M., Derhami, V.: An automatic method for cvss score prediction using vulnerabilities description. J. Intell. Fuzzy Syst. **30**(1), 89–96 (2015)
19. Elbaz, C., Rilling, L., Morin, C.: Fighting n-day vulnerabilities with automated cvss vector prediction at disclosure. In: International Conference on Availability, Reliability and Security (ARES), pp. 1–10. ACM (2020)
20. Besold, T.R., Bader, S., Bowman, H., Domingos, P., Hitzler, P., Kühnberger, K.U., Zaverucha, G.: Neural-symbolic learning and reasoning: a survey and interpretation. In: Neuro-Symbolic Artificial Intelligence: The State of the Art, pp. 1–51. IOS Press (2021)
21. Garcez, A.S., Besold, T.R., De Raedt, L., Földiak, P., Hitzler, P., Icard, T., Silver, D.L.: Neural-symbolic learning and reasoning: Contributions and challenges. In: AAAI Spring Symposium, pp. 18–21 (2015)
22. Zhang, X., Sheng, V.S.: Neuro-symbolic ai: explainability, challenges, and future trends (2024). arXiv preprint arXiv:2411.04383
23. Colelough, B.C., Regli, W.: Neuro-symbolic ai in 2024: a systematic review (2025). arXiv preprint arXiv:2501.05435
24. Tian, Z., Li, M., Sun, J., Chen, Y., Chen, L.: Enhancing vulnerability detection by fusing code semantic features with llm-generated explanations. Inf. Fusion 103450 (2025)
25. Zhequ, S., Wang, S., Lu, N., Shi, W., Liu, Z.: Mdvul: a semantic-based complex dependency code vulnerability detection using fusion path. Inf. Fusion 103475 (2025)
26. Lundberg, S., Lee, S.-I.: A unified approach to interpreting model predictions (2017). arXiv preprint arXiv:1705.07874
27. Du, D.: Optimizing the infidelity, sensitivity, and complexity of feature importance explanations for machine learning models, Harvard University, Digital Access to Scholarship at Harvard (DASH) (2025)

28. Samek, W., Wiegand, T., Müller, K.-R.: Explainable artificial intelligence: understanding, visualizing and interpreting deep learning models (2017). arXiv preprint arXiv:1705.07874
29. Mirtaheri, S.L., Movahed, N., Shahbazian, R., Pascucci, V., Pugliese, A.: Cybersecurity in the age of generative ai: a systematic taxonomy of ai-powered vulnerability assessment and risk management. Futur. Gener. Comput. Syst. **175**, 108107 (2026)
30. Aghababaei, A., Nikadon, J., Formanowicz, M., Bettinsoli, M.L., Cervone, C., Suitner, C., Erseghe, T.: Application of integrated gradients explainability to sociopsychological semantic markers (2025). arXiv preprint arXiv:2503.04989
31. Choi, C.L., Duplessis, A., Belongie, S.: Unlearning-based neural interpretations (2024). arXiv preprint arXiv:2410.08069
32. Dhaini, M., Kafaite Zahra Hussain, E., Zaradoukas, G., Kasneci: Evalxnlp: a framework for benchmarking post-hoc explainability methods on nlp models (2025). arXiv preprint arXiv:2505.01238
33. Albanese, M., Iganibo, I., Adebiyi, O.: A framework for designing vulnerability metrics. Comput. Secur. **132**, 103382 (2023)
34. Wang, J.A., Guo, M., Wang, H., Xia, M., Zhou, L.: Ontology-based security assessment for software products. In: Workshop on Cyber Security and Information Intelligence Research, pp. 1–4. ACM (2009)
35. Wang, J.A., Guo, M., Wang, H., Xia, M., Zhou, L.: Environmental metrics for software security based on a vulnerability ontology. In: 3rd International Conference on Secure Software Integration and Reliability Improvement, vol. 1, pp. 159–168 (2009)
36. Wang, J.A., Guo, M.: Ovm: an ontology for vulnerability management. In: Workshop on Cyber Security and Information Intelligence Research, pp. 1–4. ACM (2009)
37. Sammet, J., Krestel, R.: Domain-specific keyword extraction using Bert. In: Conference on Language, Data and Knowledge, pp. 659–665 (2023)
38. Sundararajan, M., Taly, A., Yan, Q.: Axiomatic attribution for deep networks (2017). arXiv preprint arXiv:1705.07874
39. Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M., Kim, B.: Sanity checks for saliency maps, 31:9505–9515 (2018). arXiv preprint arXiv:1705.07874

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.